

开发实施手册

智能体上下文系统实施方案

Redis DB 4 热上下文 + MongoDB 持久化上下文

面向现有 FastAPI / LangChain / LangGraph Agent Service

文档版本	v1.0
编制日期	2026-08-28
适用环境	Python 3.12、uv、FastAPI、LangChain、LangGraph
实施边界	本手册提供设计与示例，不自动修改项目或虚拟机

核心原则：MySQL 展示记录与模型上下文严格解耦

阅读导航

本手册按“先部署基础设施，再设计数据结构，最后接入 LangGraph”的顺序组织。建议首次阅读从第 1 章顺序进行；实施时可直接按第 14 章清单逐项验收。

- 1. 目标、边界与当前项目基线
- 2. 总体架构和数据流
- 3. Redis database 4 热上下文设计
- 4. MongoDB 虚拟机 Docker 部署
- 5. MongoDB 文档结构与索引
- 6. 上下文加载、装配与压缩
- 7. LangGraph 编排和 AgentState 扩展
- 8. Repository、依赖注入与生命周期
- 9. 接口契约与 Java 协作边界
- 10. 故障处理、安全、备份与恢复
- 11. 测试场景、实施步骤和验收清单
- 附录：配置、命令、示例文件和参考资料

本次确定的实施决策

Redis 使用现有 192.168.100.128:6379 的 database 4，热缓存 TTL 为 7 天；MongoDB 使用同一虚拟机上的 Docker 容器，固定镜像 mongo:8.0.29-noble，并作为上下文持久化真相源。第一版不接入 LangGraph checkpoint。

1. 目标、边界与当前项目基线

1.1 建设目标

把当前只能处理单轮消息的最小智能体骨架，扩展为具备独立上下文加载、压缩和持久化能力的多轮智能体，同时保持 Java 聊天业务与 Python 模型上下文之间的明确边界。

- 用户继续通过 Java 聊天接口发送消息，Java 负责认证、会话归属、MySQL 展示消息和面向浏览器的 SSE。
- Python Agent 使用 requestId、sessionId、userId 定位上下文，不接受调用方传入角色或可信历史记录。
- Redis/MongoDB 中的数据是 AgentContext，不是 MySQL ChatMessage 的副本；它可以被规范化、裁剪、总结和事实化。
- MongoDB 是持久化真相源；Redis 只是可丢失、可重建的热缓存。

1.2 当前代码基线

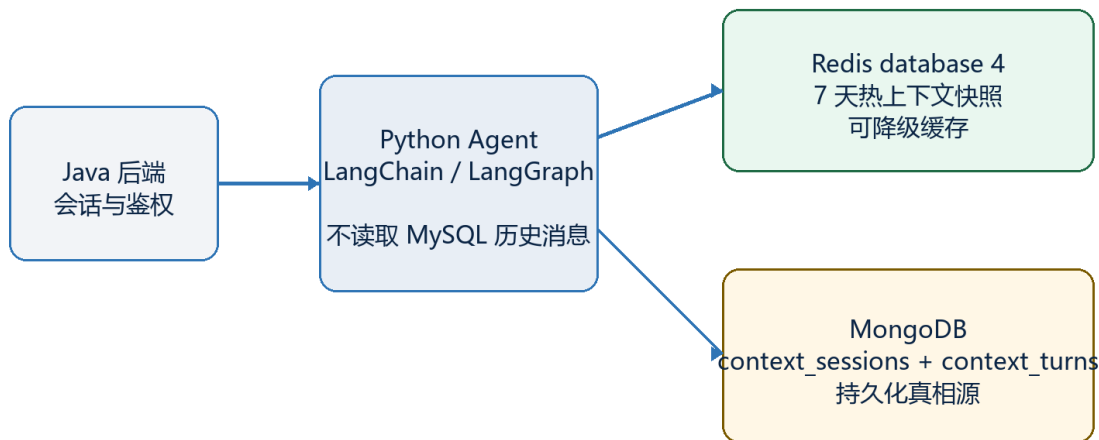
项目部分	当前状态	本阶段变化
AgentState	仅含本轮请求、角色提示词和最终答案	增加上下文快照、revision 和模型消息
LangGraph	input_guard → generate → output_validate	增加 load、compact、persist 三个节点
history	兼容 Java DTO，但不参与模型	继续保持该约束
取消机制	单进程注册表	本阶段不替换，只保证取消不持久化完整上下文
存储驱动	尚未引入 Redis/MongoDB Python 驱动	规划增加 redis 与 pymongo

1.3 明确不做的内容

- 不从 MySQL chat_message 恢复模型上下文。
- 不接入 LangGraph checkpoint，不支持中断后从图节点续跑。
- 不设计租户、角色选择接口、RAG 或业务工具调用。
- 不在本手册交付过程中修改项目源码或远程虚拟机。

2. 总体架构和数据流

Agent 上下文双层存储架构



MySQL 仅保存用户可见消息、引用和反馈；Redis/MongoDB 保存独立的模型上下文。

图 1 Agent 上下文双层存储架构

2.1 三类数据的职责

存储	保存内容	是否进入模型	故障影响
MySQL	用户提问、最终展示回答、引用、反馈	否	影响聊天业务查阅与落库
Redis DB 4	最近上下文快照、摘要、事实、revision	是	可回源 MongoDB，服务降级
MongoDB	持久快照和按请求保存的上下文轮次	是	必要依赖，失败终止上下文型聊天

为什么不能直接读取 MySQL 历史消息？

页面消息服务于用户查阅，通常包含展示格式、引用文本和失败状态；模型上下文服务于推理，需要摘要、事实、工具结论和 Token 预算。二者来源、生命周期和压缩策略不同，强行复用会让模型上下文不可控。

2.2 单轮请求的数据流

1. Java 校验登录用户和 sessionId 归属，创建 requestId 后调用 Python SSE。
2. Python 加载后端角色文件，并由 load_context 根据 userId + sessionId 读取上下文。
3. Redis 命中则刷新 TTL；未命中、损坏或不可用时从 MongoDB 恢复并尝试回填 Redis。
4. compact_context 估算角色提示词、摘要、事实、近期条目和当前问题的 Token；超过软阈值时同步压缩。
5. generate 使用装配后的消息调用模型并输出 delta；history 字段不参与装配。
6. output_validate 确认最终回答有效后，persist_context 先写 MongoDB，再刷新 Redis。
7. Java 汇总最终回答和引用写入 MySQL，并向浏览器发送最终 done。

3. Redis database 4 热上下文设计

3.1 逻辑分库说明

database 4 可以避免与 Java 使用的 database 3 在 Key 空间中混杂，但它不是物理隔离。两个逻辑库仍共享 Redis 服务的内存上限、CPU、AOF/RDB、maxmemory-policy 和故障域。当前个人项目可以采用 DB 4；如果以后需要独立资源配额，只替换连接配置即可迁移到另一个 Redis 实例。

必须保留 Key 前缀

即使已经使用 database 4，仍使用 xc:agent:context:v1: 前缀。前缀承担业务归属和结构版本作用，未来迁移、扫描、灰度和清理都不应依赖 SELECT 数据库编号本身。

3.2 Key 与 JSON 结构

Redis Key

```
|xc:agent:context:v1:session:{userId}:{sessionId}
```

热上下文快照示例

```
|{
  "schemaVersion": 1,
```

```
{
  "userId": 7,
  "sessionId": 3001,
  "revision": 12,
  "summary": "用户正在排查 UOS 打印机识别问题.....",
  "facts": [
    {"key": "os", "value": "UOS", "sourceRequestId": 101, "confidence": 1.0}
  ],
  "recentEntries": [
    {"requestId": 101, "role": "user", "content": ".....", "estimatedTokens": 38},
    {"requestId": 101, "role": "assistant", "content": ".....", "estimatedTokens": 120}
  ],
  "estimatedTokens": 1850,
  "lastRequestId": 101,
  "updatedAt": "2026-08-28T02:00:00Z"
}
```

3.3 读写和 TTL 规则

动作	Redis 行为	MongoDB 行为
加载	GET；命中后 EXPIRE 604800	Redis 未命中或失败时 find_one
首次会话	无 Key	无文档时在内存创建 revision=0 空快照
持久化	Mongo 成功后 SET EX 604800	先执行 revision 条件写入
清理	DEL，失败记录告警	删除 session 快照和全部 turns
损坏 JSON	忽略并回源，不在日志输出正文	用有效文档回填 Redis

3.4 Python 客户端生命周期

FastAPI 进程启动时创建一个共享 redis.asyncio.Redis 客户端和连接池；请求节点只复用客户端。关闭应用时调用 await redis.aclose()，避免连接泄漏。连接池默认建议 20，连接与命令超时均为 3 秒。

redis.asyncio 客户端示例

```
from redis.asyncio import Redis

redis_client = Redis(
    host=settings.redis_host,
    port=settings.redis_port,
    password=settings.redis_password.get_secret_value(),
    db=4,
    decode_responses=True,
    socket_connect_timeout=3,
    socket_timeout=3,
    max_connections=20,
)
```

```
# 应用关闭时显式释放连接池。  
await redis_client.aclose()
```

4. MongoDB 虚拟机 Docker 部署

4.1 部署结果

项目	固定值
虚拟机地址	192.168.100.128
容器名	xc-agent-mongodb
镜像	mongo:8.0.29-noble
端口	192.168.100.128:27017 → 容器 27017
数据库	xinchuang_agent_context
应用账号	agent_context_app, 仅目标数据库 readWrite
数据卷	xc_agent_context_mongodb_data

4.2 部署前检查

虚拟机检查命令

```
# 在虚拟机执行，先确认操作系统、CPU 和 Docker 环境。  
cat /etc/os-release  
lscpu | grep -i avx  
docker --version  
docker compose version  
sudo systemctl status docker --no-pager
```

MongoDB 5.0 及以上镜像要求 CPU 支持 AVX。若虚拟机未暴露 AVX，应先检查宿主机虚拟化设置，不能通过使用已停止支持的旧版本规避。

4.3 Docker 未安装时

先根据 `/etc/os-release` 选择对应官方安装说明。生产或长期开发环境应使用 Docker 官方软件仓库，不建议直接执行未知的一键脚本。

[Docker Engine for Ubuntu](#) [Docker Engine for CentOS](#)

安装后的验证

```
# Ubuntu/Debian 系安装完成后的通用验证
sudo systemctl enable --now docker
sudo docker run --rm hello-world
sudo docker compose version

# CentOS/RHEL 系同样启用服务后验证
sudo systemctl enable --now docker
sudo docker run --rm hello-world
```

4.4 创建部署目录和密码

目录与密码准备

```
sudo mkdir -p /opt/xc-agent-context/mongodb/init
sudo chown -R "$USER":"$USER" /opt/xc-agent-context
cd /opt/xc-agent-context/mongodb

# 分别生成两个不同的随机密码，不要直接使用示例密码。
openssl rand -base64 32
openssl rand -base64 32
```

把交付包中“示例文件/mongodb”目录下的 `compose.yaml`、`.env.example` 和 `init` 脚本复制到该目录；将 `.env.example` 复制为 `.env` 后替换占位值。

compose.yaml (节选)

```
# 参考实现，尚未自动接入当前项目。
services:
  mongodb:
    # 固定小版本，避免使用 latest 导致重建容器时发生不可预期升级。
    image: mongo:8.0.29-noble
    container_name: xc-agent-mongodb
    restart: unless-stopped
    environment:
      # root 账号只用于初始化和运维，Python Agent 禁止使用该账号连接。
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USERNAME}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD}
      MONGO_INITDB_DATABASE: xinchuang_agent_context
      # 以下变量由 /docker-entrypoint-initdb.d 中的脚本读取，用于创建最小权限应用账号。
      MONGO_APP_USERNAME: ${MONGO_APP_USERNAME}
      MONGO_APP_PASSWORD: ${MONGO_APP_PASSWORD}
    ports:
      # 只绑定虚拟机内网地址。还应在虚拟机防火墙中限制为开发电脑来源地址。
      - "${MONGO_BIND_IP:-192.168.100.128}:27017:27017"
```



```

volumes:
  # 命名卷保存数据库文件，删除容器不会删除数据卷。
  - agent_context_mongodb_data:/data/db
  # 初始化脚本只在数据卷首次为空时运行，并以只读方式挂载。
  - ./init:/docker-entrypoint-initdb.d:ro
healthcheck:
  # 使用容器内 root 初始化账号执行 ping; ${VAR} 用于阻止 Compose 提前展开变量。
  test:
    - CMD-SHELL
    - >-
      mongosh --quiet --host 127.0.0.1
      --username "${MONGO_INITDB_ROOT_USERNAME}"
      --password "${MONGO_INITDB_ROOT_PASSWORD}"
      --authenticationDatabase admin
      --eval "quit(db.adminCommand({ ping: 1 }).ok ? 0 : 2)"
  interval: 10s
  timeout: 5s
  retries: 10
  start_period: 20s
security_opt:
  - no-new-privileges:true

volumes:
  # 显式命名，便于备份、迁移和人工识别。
  agent_context_mongodb_data:
    name: xc_agent_context_mongodb_data

```

4.5 启动和首次验收

启动与 mongosh 验收

```

cd /opt/xc-agent-context/mongodb
docker compose --env-file .env config
docker compose pull
docker compose up -d
docker compose ps
docker compose logs --tail=100 mongodb

# 容器变为 healthy 后，使用 root 检查应用数据库和用户。
docker exec -it xc-agent-mongodb mongosh --username xc_mongo_admin --password --
authenticationDatabase admin

use xinchuang_agent_context
db.getUsers()
db.context_sessions.getIndexes()
db.context_turns.getIndexes()

```

初始化脚本只执行一次

MONGO_INITDB_* 环境变量和 /docker-entrypoint-initdb.d 脚本只在数据卷首次为空时执行。已有数据卷时修改 .env 并重启容器不会修改账号密码；应通过 mongosh 执行 changeUserPassword，或在明确无需保留数据时重建数据卷。

4.6 网络与防火墙

Compose 只把端口绑定到虚拟机内网地址 192.168.100.128。仍需在虚拟机防火墙或宿主机网络层限制来源，只允许开发电脑访问 27017，禁止映射到公网网卡。Docker 端口映射可能绕过部分 ufw 规则，应优先在 DOCKER-USER 链或上游防火墙限制。

Windows 连接验证

```
# Windows 开发电脑检查 TCP 可达性
Test-NetConnection 192.168.100.128 -Port 27017

# 安装 mongosh 后使用应用账号验证最小权限连接
mongosh "mongodb://agent_context_app:<URL 编码密码>@192.168.100.128:27017/xinchuang_agent_context?authSource=xinchuang_agent_context" --eval
"db.runCommand({ ping: 1 })"
```

5. MongoDB 文档结构与索引

5.1 context_sessions

每个 userId + sessionId 只有一份当前快照。该集合是模型装配的持久化来源，Redis Key 过期后可直接恢复。revision 从 0 递增，用于乐观并发控制。

context_sessions 示例

```
{
  "schemaVersion": 1,
  "userId": 7,
  "sessionId": 3001,
  "revision": 12,
  "summary": "用户正在排查 UOS 外设问题.....",
  "facts": [{"key": "os", "value": "UOS", "sourceRequestId": 101}],
  "recentEntries": [/* AgentContextEntry, 不是 ChatMessage */],
  "estimatedTokens": 1850,
  "lastRequestId": 101,
  "createdAt": {"$date": "2026-08-28T01:00:00Z"},
  "updatedAt": {"$date": "2026-08-28T02:00:00Z"}
}
```

5.2 context_turns

每个 requestId 保存一轮用于重建和诊断的上下文内容。userContext 和 assistantContext 可以来自本轮文本的规范化结果，不要求与 MySQL 展示内容逐字一致。

context_turns 示例

```
{
  "requestId": 101,
  "userId": 7,
  "sessionId": 3001,
  "userContext": "用户反馈 UOS 无法识别 USB 打印机。",
  "assistantContext": "建议确认 USB 枚举、打印服务和驱动兼容性，并按顺序排查。",
  "modelName": "actual-model-name",
  "createdAt": {"$date": "2026-08-28T02:00:00Z"}
}
```

5.3 索引和幂等

集合	索引	用途
context_sessions	unique(userId, sessionId)	保证一个会话一个快照
context_turns	unique(requestId)	Java 重试不会重复写入轮次
context_turns	userId + sessionId + createdAt	按会话重建和诊断

快照写入使用 userId、sessionId、expected revision 作为条件；命中后写 revision+1。若 matchedCount=0，先检查 lastRequestId 是否等于当前 requestId：相等表示此前已成功，按幂等成功返回；否则返回 CONTEXT_REVISION_CONFLICT。

6. 上下文加载、装配与压缩

6.1 模型消息装配顺序

顺序	消息类型	内容来源
1	SystemMessage	活动角色文件 systemPrompt
2	SystemMessage	Mongo/Redis 会话摘要
3	SystemMessage	已确认事实与用户约束
4	Human/AI Messages	近期 AgentContextEntry

顺序	消息类型	内容来源
5	HumanMessage	当前请求 message

禁止来源

ChatStreamRequest.history、MySQL chat_message、浏览器缓存和未经校验的调用方 systemPrompt 均不得直接进入模型上下文。

6.2 Token 预算

参数	默认值	含义
chars_per_token	2.0	中英文混合文本的保守近似
soft_token_limit	8000	达到时触发同步压缩
hard_token_limit	12000	最终输入不得超过该预算
recent_turns	6	压缩后保留最近六轮原文
summary_max_tokens	1200	摘要模型最大输出

预算必须同时计算系统提示词、摘要、事实、近期消息和当前问题。max_output_tokens 只约束模型输出，不能代替输入上下文预算。若当前问题本身已经超过硬预算，应在调用模型前返回 INPUT_TOO_LARGE。

6.3 同步压缩算法

- 1. 计算装配后消息 Token，未超过 8000 时跳过压缩。
- 2. 从快照中分离最近六轮，较旧条目作为待总结材料。
- 3. 压缩提示词要求保留用户明确目标、约束、已确认事实、未完成事项和关键工具结论。
- 4. 模型输出结构化 summary、facts 和 invalidatedFacts，Pydantic 校验后生成新快照。

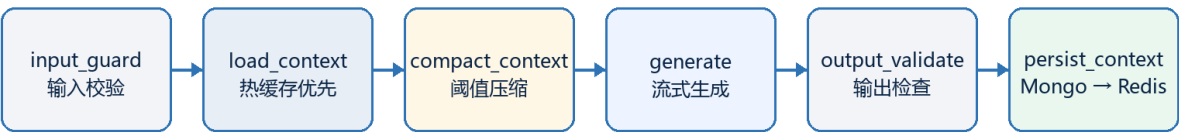
- 5. 再次计算 Token；若仍超过 12000，从最旧近期条目开始确定性裁剪，不删除角色提示、摘要和事实。
- 6. 压缩结果先写 MongoDB、再写 Redis，然后继续正式回答。即使本轮回答被取消，已经完成的历史压缩仍然有效。

6.4 压缩失败兜底

摘要模型超时、输出 JSON 无效或结果超过预算时，不把异常原文暴露给 Java。第一版使用确定性裁剪：保留旧摘要与事实，按时间从旧到新移除近期条目，直到低于硬预算；同时记录 context_compaction_fallback 指标。若仅当前问题已经超限，则不能裁剪用户当前问题，应终止请求。

7. LangGraph 编排和 AgentState 扩展

第一版 LangGraph 上下文编排



异常原则：Redis 失败回源 MongoDB；MongoDB 必要读写失败则终止本轮；取消或断开不写完整上下文。

图 2 第一版 LangGraph 上下文编排

7.1 节点职责

节点	职责	允许写入的状态
input_guard	规范化当前输入、做长度与格式检查	message

节点	职责	允许写入的状态
load_context	Redis 优先, Mongo 回源, 记录 revision	context_snapshot、context_revision
compact_context	估算与同步压缩, 构造模型消息	context_snapshot、model_messages
generate	使用 model_messages 流式调用模型	final_answer
output_validate	确认可展示回答非空且通过规则	校验结果
persist_context	Mongo CAS + turn 幂等写入, 再刷新 Redis	新 revision

7.2 AgentState 建议字段

状态结构示意

```
class AgentState(TypedDict):
    request_id: int
    session_id: int
    user_id: int
    message: str
    role_name: str
    system_prompt: str
    max_output_tokens: int
    context_snapshot: ContextSnapshot
    context_revision: int
    model_messages: list[BaseMessage]
    final_answer: str
```

Redis、MongoDB、模型客户端和 Repository 不能放入 AgentState。它们是进程级基础设施，应通过 AgentRuntime 闭包或服务容器注入节点，保持状态可序列化并便于测试。

7.3 终态和取消

- persist_context 必须位于 output_validate 之后，因此空回答不会写入上下文。
- 取消标记在模型流中被检测到后抛出 AgentCancelledError，不执行 persist_context。
- 客户端断开触发 runtime.cancel；event_generator return 后 AgentRuntime.finally 清理活动运行。
- 如果 MongoDB 在模型已经输出 delta 后写入失败，SSE 以 error 终止，Java 不应把该次回答标记为 COMPLETED。

8. Repository、依赖注入与生命周期

8.1 依赖选择

依赖	用途	选择原因
redis	redis.asyncio 热缓存	与 FastAPI 异步事件循环一致
pymongo	AsyncMongoClient 持久化	官方异步 API；不采用即将弃用的 Motor
Pydantic	Redis/Mongo 文档校验	现有项目已使用，保证 schemaVersion 边界
LangChain Core	消息和近似 Token 计数	复用现有消息抽象

pyproject.toml 依赖示意

```
dependencies = [  
    # .....现有依赖.....  
    "redis>=6.4.0,<7.0.0",  
    "pymongo>=4.13.0,<5.0.0",  
]
```

8.2 FastAPI lifespan

应用启动时创建客户端、执行 MongoDB ping 和索引检查，并将 ContextRepository 注入 AgentRuntime。MongoDB 探测失败时服务 readiness 不通过；Redis 探测失败只标记 DEGRADED。应用关闭时依次关闭 Redis 和 AsyncMongoClient。测试通过参数注入内存 Repository，不访问开发者数据库。

生命周期示例

```
@asynccontextmanager  
async def lifespan(app: FastAPI):  
    redis_client = create_redis_client(settings)  
    mongo_client = AsyncMongoClient(settings.mongodb_uri)  
    await mongo_client.admin.command("ping")  
    repository = ContextRepository(redis_client, mongo_client, context_settings)  
    app.state.agent_runtime = create_runtime(repository=repository)  
    try:  
        yield  
    finally:  
        await redis_client.aclose()  
        await mongo_client.close()
```

8.3 仓储错误分类

内部异常	SSE code	retryable	处理
ContextStoreUnavailableError	CONTEXT_STORE_UNAVAILABLE	true	Mongo 必要读写失败，终止
ContextRevisionConflictError	CONTEXT_REVISION_CONFLICT	true	重新加载后可重试一次
ContextDataInvalidError	CONTEXT_DATA_INVALID	false	隔离损坏数据并人工检查
ContextTooLargeError	INPUT_TOO_LARGE	false	要求缩短当前问题或附件

9. 接口契约与 Java 协作边界

9.1 现有聊天流接口

POST /internal/ai/v1/chat/stream 的请求结构保持兼容。requestId、sessionId、userId 继续由可信 Java 服务传入；history 允许缺省、空数组或 null，但上下文节点明确忽略它。

内部 SSE 可新增 stage=context 的 status 事件。Java 对外转发时可显示“正在加载上下文/正在压缩上下文/正在保存上下文”，但不要把些临时状态当作 MySQL 模型上下文。若当前 chat_message.stage 数据库约束不包含 context，Java 应只转发而不直接写该枚举，或另行执行数据库迁移。

9.2 清理上下文接口

方法与路径

```
|DELETE /internal/ai/v1/context/sessions/{sessionId}?userId={userId}
```

参数	位置/类型	约束与含义
sessionId	path / int64	必填，>0，对应 chat_session.id
userId	query / int64	必填，>0，由 Java 从认证上下文取得
X-Internal-Token	header / string	内部鉴权开启时必填

200 响应示例

```
{
  "sessionId": 3001,
  "userId": 7,
  "status": "CLEARED",
  "deleted": {
    "mongoSessions": 1,
    "mongoTurns": 18,
    "redisKeys": 1
  }
}
```

未找到任何数据时仍返回 200，status=NOT_FOUND，所有删除数量为 0。MongoDB 清理失败返回 503；Redis 删除失败但 MongoDB 已清理时返回 200，并在日志与指标中记录 Redis 残留，最多保留到七天 TTL。

9.3 Java 删除会话后的协作

1. Java 先用 sessionId + 当前 userId 完成 MySQL 软删除事务。
2. 事务提交后调用 Python 清理接口，不把 Python 调用放进 MySQL 数据库事务。
3. 清理调用失败不恢复用户可见会话；Java 记录待重试任务和 requestId，后台按指数退避重试。
4. Python 清理接口幂等，因此 Java 可以安全重复调用。

9.4 健康检查

Redis 故障时的非敏感健康信息

```
{
  "status": "DEGRADED",
  "service": "xinchuang-agent-service",
  "mongodb": "UP",
  "redis": "DOWN",
  "contextAvailable": true
}
```

MongoDB DOWN 时 contextAvailable=false，readiness 应失败；Redis DOWN 时 status=DEGRADED 但 contextAvailable=true。健康接口不得返回连接 URI、用户名、密码、数据库正文或供应商异常原文。

10. 故障处理、安全、备份与恢复

10.1 故障矩阵

场景	系统行为	用户/SSE 结果
Redis 读取超时	记录告警，回源 MongoDB	继续回答，可显示降级状态
Redis 写入失败	Mongo 已成功，跳过缓存	done 正常；下一轮回源
Mongo 加载失败	不调用正式回答模型	error: CONTEXT_STORE_UNAVAILABLE
Mongo 保存失败	不发送 done，Java 不标记 COMPLETED	已发 delta 后以 error 终止
Redis JSON 损坏	忽略 Key，Mongo 回源并覆盖	继续回答
revision 冲突	重新加载后重试一次，仍冲突则终止	error: CONTEXT_REVISION_CONFLICT
压缩模型失败	确定性裁剪，记录 fallback 指标	预算满足后继续
客户端取消/断开	中断模型，不写完整本轮上下文	RUN_CANCELLED 或直接断开

10.2 安全要求

- MongoDB root 和应用账号使用不同的至少 24 位随机密码；Agent 只使用 readWrite 应用账号。
- 密码通过环境变量或密钥文件注入，日志、Trace、异常响应和 Git 中不得出现明文。
- MongoDB 只绑定内网 IP，防火墙限制来源；需要跨不可信网络时再启用 TLS。
- 上下文正文可能包含个人信息，日志只记录 requestId、sessionId、耗时、命中来源和计数，不记录 content。
- 内部清理接口继续使用现有 Token；生产接入 Java 前升级为 TLS + HMAC + nonce 防重放。
- 备份文件与数据库同等敏感，应加密存储并限制访问权限。

10.3 备份与恢复

备份与恢复命令

```
# 在虚拟机执行逻辑备份。密码通过交互输入或受保护变量提供。
docker exec xc-agent-mongodb mongodump --username xc_mongo_admin --authenticationDatabase admin --db xinchuang_agent_context --archive=/tmp/agent-context.archive --gzip

docker cp xc-agent-mongodb:/tmp/agent-context.archive ./agent-context.archive

# 恢复前先在隔离环境演练；--drop 会覆盖目标集合，禁止在未确认目标时执行。
docker cp ./agent-context.archive xc-agent-mongodb:/tmp/agent-context.archive
docker exec -it xc-agent-mongodb mongorestore --username xc_mongo_admin --authenticationDatabase admin --archive=/tmp/agent-context.archive --gzip
```

Redis 不作为备份对象。MongoDB 恢复完成后，Redis DB 4 可以自然回源重建；如需强制清空旧缓存，应使用明确前缀的 SCAN + DEL 工具，禁止对共享 Redis 执行 FLUSHALL。

10.4 监控指标

指标	意义
context_load_total{source}	区分 redis、mongodb、empty 的加载次数
context_load_duration_ms	上下文加载耗时
context_compaction_total{result}	压缩成功、跳过和 fallback 次数
context_tokens_before/after	压缩前后 Token 规模
context_persist_duration_ms	Mongo + Redis 持久化耗时
context_storage_error_total{store,operation}	按存储和操作统计异常
context_revision_conflict_total	同会话并发写冲突

11. 测试场景、实施步骤和验收清单

11.1 单元测试

场景	准备	断言
首次会话	Redis/Mongo 均无数据	生成 revision=0 空上下文，不访问 MySQL
Redis 命中	存在合法 JSON	使用缓存并刷新 7 天 TTL
Redis 未命中	Mongo 有快照	回源并回填 Redis
Redis 故障	命令抛 RedisError	使用 Mongo，聊天继续
Mongo 故障	find/replace 抛异常	CONTEXT_STORE_UNAVAILABLE
压缩阈值	估算 Token > 8000	调用压缩器，结果 <= 12000
压缩失败	压缩器超时或 JSON 无效	确定性裁剪并记录 fallback
重复 requestId	相同请求保存两次	Mongo turn 只有一条，revision 不重复增加
revision 冲突	快照已被其他请求更新	重试一次或返回冲突
取消	生成中设置取消标记	不调用 persist_context
history=null	Java 显式传 null	请求成功且 history 不进模型
清理幂等	连续调用两次 DELETE	第二次返回 NOT_FOUND

11.2 集成验收

- docker compose ps 显示 MongoDB healthy；应用账号可以访问目标库但不能执行 admin 管理操作。
- Redis DB 3 原有 Key 不变；Agent 上下文只出现在 DB 4 且带 xc:agent:context:v1: 前缀。
- 第一轮聊天后 MongoDB 两个集合和 Redis 均出现对应 sessionId；MySQL 仍只保存展示消息。
- 删除 Redis Key 后继续提问，Agent 从 MongoDB 恢复且自动回填 Redis。

- 临时停止 Redis 后聊天仍可完成，健康状态为 DEGRADED。
- 临时停止 MongoDB 后新聊天在模型生成前返回 CONTEXT_STORE_UNAVAILABLE。
- 构造超过软阈值的上下文后出现压缩状态，Mongo revision 增加，模型输入低于硬预算。
- Java 软删除会话后调用清理接口，Mongo 文档和 Redis Key 均被删除。

11.3 推荐实施顺序

1. 在虚拟机部署 MongoDB，完成账号、索引、网络和备份验收。
2. 向 pyproject.toml 增加 redis、pymongo，更新 uv.lock。
3. 扩展 Settings 和 .env.example，所有密码使用 SecretStr。
4. 实现 Pydantic 上下文模型和 ContextRepository 协议；先完成内存版本单元测试。
5. 实现 Redis/Mongo Repository，增加 lifespan 创建、ping、关闭和测试注入。
6. 扩展 AgentState，加入 load_context、compact_context、persist_context 三个节点。
7. 扩展 ModelGateway 的非流式摘要能力和 Mock 压缩器。
8. 补充上下文异常到 SSE error 的映射以及依赖健康状态。
9. 新增内部清理接口，再接入 Java 会话删除后的重试调用。
10. 完成故障注入、并发、取消、压缩和端到端验收后再开启真实模型。

11.4 回滚策略

第一版应增加 AGENT_CONTEXT_ENABLED 开关。回滚时关闭上下文功能，LangGraph 回到 input_guard → generate → output_validate；不要删除 Mongo 数据卷。确认新版本稳定后再决定是否清理旧 schemaVersion 数据。Redis DB 4 缓存可自然过期，不执行 FLUSHALL。

12. 配置清单

Agent 上下文环境变量模板

```
# 参考实现，尚未自动接入当前项目。
# 可把这些变量合并到 agent_service/.env；真实密码不得提交到 Git。
```

```
# Redis 热上下文：沿用虚拟机 Redis，但固定使用 database 4。
AGENT_REDIS_HOST=192.168.100.128
AGENT_REDIS_PORT=6379
AGENT_REDIS_PASSWORD=<本地 Redis 密码>
AGENT_REDIS_DATABASE=4
AGENT_REDIS_CONNECT_TIMEOUT_SECONDS=3
AGENT_REDIS_SOCKET_TIMEOUT_SECONDS=3
AGENT_REDIS_MAX_CONNECTIONS=20
AGENT_CONTEXT_REDIS_TTL_SECONDS=604800
AGENT_CONTEXT_REDIS_KEY_PREFIX=xc:agent:context:v1:

# MongoDB 是上下文持久化真相源。URI 中的特殊字符必须进行 URL 编码。
AGENT_MONGODB_URI=mongodb://agent_context_app:<URL 编码后的应用密码>@192.168.100.128:27017/xinchuang_agent_context?authSource=xinchuang_agent_context
AGENT_MONGODB_DATABASE=xinchuang_agent_context
AGENT_MONGODB_SERVER_SELECTION_TIMEOUT_MS=3000
AGENT_MONGODB_CONNECT_TIMEOUT_MS=3000

# 上下文预算与压缩策略。
AGENT_CONTEXT_SOFT_TOKEN_LIMIT=8000
AGENT_CONTEXT_HARD_TOKEN_LIMIT=12000
AGENT_CONTEXT_RECENT_TURNS=6
AGENT_CONTEXT_SUMMARY_MAX_TOKENS=1200
AGENT_CONTEXT_CHARS_PER_TOKEN=2.0
```

12.1 配置校验规则

- AGENT_REDIS_DATABASE 固定默认为 4，范围为 0 ~ 15；生产若禁用逻辑库，应改为独立实例地址。
- Mongo URI 必须包含目标数据库和 authSource，不允许使用 root 账号作为应用连接。
- soft_token_limit 必须小于 hard_token_limit；recent_turns 至少为 1。
- 连接和命令超时必须大于 0；Redis TTL 不低于 60 秒。
- production 环境启用 context 时，Mongo URI 和 Redis 密码不得为空。

13. 示例文件与接入说明

交付包中的 Python 文件不是直接复制到一个目录即可运行的独立服务，而是与当前 agent_service 代码结构对齐的参考实现。正式接入时应把数据模型、基础设施仓储和 LangGraph 节点分别放到项目对应层，并通过 AgentRuntime 注入。

示例文件	建议接入位置	用途
------	--------	----

示例文件	建议接入位置	用途
context_models.py	schemas/context.py	Pydantic 上下文模型
context_repository.py	services/context_repository.py 或 infrastructure/context	Redis/Mongo 双层仓储
context_nodes.py	graph/nodes/context.py	load、compact、persist 节点
.env.context.example	合并进 agent_service/.env.example	运行配置说明
compose.yaml + init	复制到虚拟机独立目录	MongoDB 部署与初始化

14. 最终验收清单

- [] MongoDB 容器使用固定镜像版本并显示 healthy。
- [] 应用账号只有 xinchuang_agent_context 的 readWrite 权限。
- [] context_sessions 和 context_turns 的三个索引存在。
- [] Agent Redis 配置使用 database 4 和版本化 Key 前缀。
- [] Redis 读取/写入故障不会阻断 Mongo 可用情况下的聊天。
- [] MongoDB 故障不会静默退化成无上下文回答。
- [] ChatStreamRequest.history 没有进入模型消息。
- [] MySQL chat_message 没有被 Python Agent 用于上下文恢复。
- [] Token 超过软阈值会同步压缩，最终低于硬预算。
- [] 重复 requestId 不会重复增加 revision 或插入 turn。
- [] 取消、断开、空模型输出不会保存完整本轮上下文。
- [] 会话清理接口幂等，并由 Java 在软删除后重试调用。
- [] 日志、Trace、SSE 和 Git 中没有真实密码或上下文正文。
- [] MongoDB 备份可以在隔离环境完成恢复演练。

附录 A：官方参考资料

- [Redis redis-py 异步操作](#)
- [PyMongo Async 迁移与 Motor 弃用说明](#)
- [PyMongo AsyncMongoClient 连接说明](#)
- [Mongo Docker Official Image](#)
- [MongoDB Community Docker 安装说明](#)
- [Docker Engine Ubuntu 安装说明](#)
- [Docker Engine CentOS 安装说明](#)

附录 B：关键决策摘要

决策	选定方案	理由
Redis 隔离	同实例 database 4 + Key 前缀	满足当前个人项目逻辑隔离
Redis TTL	7 天滑动过期	活跃会话保持热，过期可从 Mongo 恢复
Mongo 部署	虚拟机 Docker 单节点	本机无需安装，部署和备份清晰
Python Mongo 驱动	PyMongo Async	适配 FastAPI，避免采用 Motor
压缩时机	软阈值触发同步压缩	一致性优先，压缩只偶尔增加首字延迟
故障策略	Mongo 必需、Redis 可降级	保证上下文不静默丢失
图持久化	第一版不接 checkpoint	控制复杂度，先完成上下文闭环